Generating YARA Rules for Malicious PyPI Packages with Small, Local LLMs

Abstract

Writing YARA rules by hand is labour- and time-intensive even for expert analysts, and does not scale with the number of malicious packages being published (Raff et al., 2020), although malicious packages are published to the Python Package Index (PyPI) which is a persistent threat to the software supply-chain. While Large Language Models (LLMs) have the possibility to assist in automating the rule generation process, the use of cloud-based LLMs for analyzing malware introduces data-governance challenges due to the need to send code to a third-party service provider. The aim of this work, then, is to investigate whether useful YARA rules can be created for detecting malicious PyPI packages by small, locally-run LLMs. Five open models with different number of parameters (from 1B to 20B) were requested to produce rules for 15 malicious behaviour categories and the rules produced were compared with a set of hand-written rules and evaluated on a set of malicious and benign packages. Validation and compilation of rules were more dependent on the model being tuned for code than on the size of the model, with the code-tuned models performing best. The rules generated were able to capture a significant percentage of malicious packages, but did not capture the hand-written rules, which had more detection and fewer false positives. False positives were diminished with the use of more detailed prompts. These findings indicate that small local LLMs can assist in rule writing but cannot yet be used in place of hand-written rules; and that their generated rules require human expertise for review.

Section 1

1. Introduction

Being able to detect malicious packages at scale is therefore a current security concern, with recent incidents including the compromise of the PyTorch Lightning and Intercom-client packages to steal credentials, and of Microsoft's durabletask package, all reported in 2026. YARA is one common detection method, which can be used to detect files that match with text strings and logical conditions. It takes expertise and manual work to write good YARA rules and it's not keeping up with the large number of malicious packages being published (Raff et al., 2020). In recent years, rules have been generated automatically using Large Language Models (LLMs), but most of them have been based on large cloud-based models.

The cloud-based models bring up data-governance issues. Analysing malware using a cloud model requires sending the malware, and possibly sensitive material from the environment being

analysed, to a third party organization, which relinquishes control over where the material is stored, processed, and analysed. For many security teams this is restricted or not allowed. This does not happen when the analyst runs his own LLM, as the data stays on his own machine. Running models locally, on the other hand, restricts the size of the models, as the biggest cloud models cannot be run on local hardware. The question is then, can the smaller models that are suitable for local use do the job at all?

In this report, we investigate the feasibility of creating a set of useful YARA rules using small, locally-run LLMs for the detection of malicious PyPI packages. Five open models were locally evaluated with Ollama and their rules were extracted for 15 behaviour categories derived from an existing taxonomy (Guo et al., 2023). A collection of malicious and benign packages were evaluated using the rules, with a set of hand-written rules as a baseline. The report aims to answer the following three research questions:

- RQ1. In what ways are a local LLM's capacity to produce valid and effective YARA rules influenced by its capability, particularly its size and tuning?
- RQ2. How does the level of detail in the prompt affect the quality of the generated rules?
- RQ3. To what extent are the generated rules able to catch malicious packages, compared to a hand-written baseline?

This work's contribution is an empirical study of the use of locally deployed small LLMs on the effect of model size, tuning and prompt detail for YARA rule generation. The above-mentioned factors have not been investigated in combination in previous studies, and previous studies have only attempted to generate rules from malware samples based on a single cloud-hosted model (Zhang, X. et al., 2025). The results are only applicable to the data set analyzed, but demonstrate the potential and limitations of the method.

The rest of the report is organised as follows. The related work is addressed in Section 2. The methodology is described in section 3. The design is presented in Section 4, and the implementation in Section 5. The results are reported and discussed in Section 6. Section 7 is a conclusion and further research is suggested.

Section 2

2. Related Work

This section summarizes existing research on three topics: Malicious package detection, automatic YARA rule generation, and applying LLMs to security tasks. It concludes with highlighting the gap/missing link the research questions attempt to fill.

2.1 Detecting Malicious Packages

The primary source for this work is Guo et al. (2023) who did a large study on malicious code in the PyPI ecosystem and created a dataset of malicious packages, which they then categorized into a taxonomy of suspicious APIs, malicious behaviours and evasion techniques. It is strong in the sense that it gives a data set and a taxonomy of behaviour that has been further developed by later work such as this study. However, it has a drawback: This addresses malicious behavior, but it doesn't include a detection tool. Ladisa et al. (2023b) have organised the wider range of supply-chain threats in their taxonomy, which lists more than a hundred different types of attacks, with real incidents mapped to them. This can help to state the problem, but as a survey, it does not detect.

There have been a number of detection methods since. The lightweight, fast and practical Amalfi classifier by Sejfia and Schäfer (2022), on the other hand, is weak against new or obfuscated attacks, as it has a fixed set of features. In Zhang et al (2025), authors modeled malicious behavior as a sequence and used a single model for detecting packages on both npm and PyPI, though as with many learned classifiers, the model is difficult to interpret. Malicious packages can be identified using code clustering, where outliers are identified without relying on labeled data, but its definition of "normal" relies on the model.Liang et al. (2023) used code clustering instead of using labeled data, treating malicious packages as outliers, which is dependent on the definition of "normal". Another is Mal-Wukong, a fast multilingual detector by Li et al. (2023) which has a wide scope, but requires constant maintenance to follow the new trends. Cross-language detection was also explored by Ladisa et al. (2023a) with cross-language features being accurate across both ecosystems npm and PyPI, but accuracy being slightly compromised per-ecosystem. Gao et al. (2025) propose MalGuard which relies on the concept that a well-designed feature set makes the lightweight models competitive with LLM based ones. In PyPIMalDet, Yan & Zhao (2025) were able to incorporate code and metadata characteristics for the improvement of accuracy; however, this has resulted in a complicated system. The majority of these generate classifiers which do not include a human-readable and deployable signature for the decision, like a YARA rule.

The automated generation of YARA rules is covered in 2.2.

Writing YARA rules can be a very time consuming exercise and a few works have attempted to automate it. Raff et al. (2020) developed a n-gram based biclustering algorithm to quickly produce rules on low-end machines: AutoYara. It is fast but as the authors point out, it is the limitation that automatic rules tend to be less effective than hand-written rules and are dependent on having a large database of fixed n-grams. The latter was extended by Mansour et al. (2025) who used deep language modelling and learning-to-rank to select more effective n-grams, but at the expense of increased complexity. While built by Coscia et al., APIARY generates rules from API patterns and shows good results, it assumes the presence of Windows executables instead of source-code ecosystems, such as PyPI. These methods rely on statistically or structurally based properties, but not necessarily knowledge of what makes code bad - this can be provided by the LLM.

2.3 LLMs in Security and Rule Generation

There has been a burgeoning field of research that brings LLMs to security.More and more research has focussed on bringing LLMs to security. Patsakis et al. (2024) evaluated the capabilities of LLMs to deobfuscate real malware and observed that models that can produce them successfully can also hallucinate and be inconsistent. Zahan et al. (2025) created SocketAI, an LLM workflow with reported high accuracy to detect malicious npm packages, using large cloud based models. To tackle this challenge, Toda and Mori (2025) have suggested a system named CHASE that relies on multiple coordinated LLM agents with tools for the analysis of malicious PyPI packages, albeit at a significant expense of analysis time.

The most similar study is that of Zhang, X. et al. (2025) which generates YARA and Semgrep rules via an LLM and obtains high precision and recall on malicious packages. It is different from this study in two aspects which justify the present study. Firstly, it is based on real malware code, meaning that it can end up with rules that are similar to the code of the malware, but not generalising. Secondly, it employs only one large model and it does not examine the impact of the size of the models, tuning or detail of the prompts on the result. Finally, and relevant to the use of small local models, Levi et al (2026) demonstrated that, for security tasks, a tuned set of 4B to 20B parameters can outperform or match much larger models, suggesting that small models are a viable choice and small model tuning is more important than size.

The summary of the literature and research gap is given in the following section.

The reviewed papers highlight that detection of malicious packages is an ongoing challenge; automated YARA generation is helpful, but that the quality of rules generated is not as strong as hand-crafted rules; and that LLMs introduce a beneficial capability of reasoning about code. Current LLM-based rule generation suffers from the drawback of using large cloud-based models which brings up the data-governance issue outlined in Section 1, and from basing generation on specific samples using a single model, without studying the impact of model size, tuning, and prompt level of detail. There is no existing work which investigates whether small, local models can learn effective rules from behaviour descriptions instead of examples, or the impact of these factors on the performance of the small, local models. This gap is the motivation for the research questions.

3. Research Methodology

In this study, locally-hosted LLMs are used to generate YARA rules which are then tested on a labelled set of malicious and benign packages. This section provides a description of the dataset, the inputs to the models, the models and prompts, and evaluation of the rules.

3.1 Dataset

The dataset consists of the set of malicious PyPI packages published by Guo et al. (2023). The malicious packages are packaged as compressed archives and were stored in compressed form and needed to be unpacked prior to analysis, since scanning for a compressed archive will perform a

pattern match against the compressed bytes not the actual source code. Following extraction, there were 10,179 malicious packages, which had 145,261 files, among which 37,628 files were source files in Python. 4,682 popular packages that are widely used on PyPI were selected as a benign negative class, by downloading the most downloaded packages which are unlikely to be malicious. This yields a class size ratio of about 2:1.

Detection is counted per package: A package is considered detected if a file contained in the package matches any of the rules. This is because the main reason why a malicious package has one malicious file in the mixture of ordinary ones is that, counting per file, would underestimate the detection.

3.2 Behaviour Specifications

The rules were written based on descriptions of behaviour instead of actual pieces of malware, to prevent rules that match only the specific pieces of malware that the rules were based on. The 15 behaviour categories were derived from the taxonomy of Guo et al. (2023), and consist of suspicious APIs (e.g., network use, file use, process use); malicious behaviours (e.g., command execution, information stealing); evasion techniques (e.g., trace erasure, platform targeting). They were both short descriptions of the behaviour, neither of which mentioned any API or strings, so the model had to determine the indicators. Thus these descriptions were hard-coded before generation, and reused across all models and prompts, making any variations in the output solely due to the model and/or prompt.

3.3 Models

The selection of models was determined by the central idea of this study: running on the local hardware, which is the one that is the motivation of the data-governance problem, (Zhang, X. et al., 2025). Cloud-based models must be sent to a third-party institution to be analysed, so only models that could be located locally by running them with Ollama on a CPU-only machine were considered, meaning a practical limit of about twenty billion parameters. This is a direct result of studying the problem instead of a side constraint because, by definition, large cloud models (Zhang, X. et al., 2025) have been used for previous rule generation, which are not applicable for deployment locally.

The models chosen from the pool of models that could be run locally include five models that have sizes from one billion up to twenty billion parameters. These five models consist of both the general-purpose and code-specific fine-tuning models. The composition of the models enables us to make two types of comparisons, one among models of various sizes in the same family and another comparison among models of the same size family.

The study of tuning is driven by new research showing that domain-specific, smaller models can perform as well or better than large, general models on security tasks (Levi et al., 2026). Wider set

have different family and training data so trends across all 5 are interpreted as indicative, the two controlled sets are given the weight of the size and tuning comparisons. **Llama 3.2 (1B)** is a small general purpose model from the Llama family of models from Meta, which has around 1 billion parameters. It is one of the lightest in the line and is designed for a light load application. It's the smallest sized one in terms of size and is not code-tuned.

Llama 3.1 (8B) is a bigger general purpose Llama model, having approximately 8 billion parameters. It is also not code-tuned, and comes from Meta. It offers a size comparison against the 1B model, and a tuning comparison against the similarly-sized code model below.

Qwen2.5-Coder (7B) is a code-specialised model of the Qwen family, having approximately seven billion parameters. Unlike the general-purpose models, it is tuned specifically for code. It has a similar scale to that of Llama 3.1 (8B) to make it possible to compare code tuning with general-purpose tuning.

Qwen2.5-Coder (14B) is a code-specialised version with approximately fourteen billion parameters. It is the family and code tuning that are common with the 7B model; it also differs primarily in size, thus the 7B versus 14B size comparison is the most obvious of the bunch.

Gpt-oss (20B) The largest model that is used is which contains approximately twenty billion parameters, and is general purpose. It is the upper end of the size range and can be used to determine if there is an improvement with size in the absence of code tuning.

This set will enable 2 comparisons to be made, where only one variable is changed. We compare size (within the same family) with tuning fixed, between Qwen2.5-Coder 7B and 14B. The general-purpose Llama 3.1 (8B) and code-tuned Qwen2.5-Coder (7B) are tuned at a comparable size and compared. In all five models, family, training data and tuning vary simultaneously and so any general trend is deemed to be suggestive rather than conclusive.

3.4 Prompts

No existing prompt set for YARA rule generation from behavioural description was found in the literature, with the closest previous work focused on YARA rule generation from malware samples, and the prompts were not written specifically for this purpose (Zhang et al., 2025). To answer this second research question, which looked at the impact of prompt detail on rule quality, more than one prompt was needed, so three levels of prompt detail were designed to determine if increasing prompt detail affects the quality of the rules. The prompts are cumulative so that every question gives a specific level of direction on the last prompt. The baseline provides only a description of the behaviour and a request for a rule.

The second includes an instruction to target behaviour common to multiple malware families, instead of just one sample, which is the commonly observed phenomena of rules based on a single sample to overfit that sample's incidental indicators and not generalise to variants (Canfora et al.,

2020; Coscia et al., 2025). The third introduces the concept of detection-oriented guidance, which consists in combining multiple conditions from several co-occurring indicators, and in favouring precision over recall, as proposed by the YARA rule quality literature (which states that the looseness of a rule is decreased by requiring multiple indicators to co-occur in the condition rather than matching on a single generic pattern; Canfora et al., 2020; Bilot et al., 2026). The behavioural content (behaviour name, type and description) is the same in all three prompts and only the surrounding guidance is different, which means that differences in the quality of the output produced between the two prompts that are next to each other can only be due to the level of guidance provided by the single layer of guidance added to the one prompt. The prompts were not derived from a formally validated template, but rather were developed by the researcher; the prompts will be considered to be a controlled instrument for testing the effect of prompt detail, not an exhaustive list of possible prompt strategies.

No existing prompt set for prompting the process of generating YARA rules from behavioural descriptions was found in the literature reviewed, with the closest to this work being based on malware samples instead of behavioural descriptions (Zhang et al., 2025), and their prompts were not designed for this task. The second research question (How does the use of a prompt for detail impact rule quality?) cannot be tested with a single prompt, so three prompts of varying levels of guidance were created. Each prompt adds one identifiable level of direction/facilitation on the previous one. The baseline consists of a description of the behaviour, and a request for a rule. The second is an instruction-based rule configured to target behaviour observed across multiple malware families rather than a single sample, which is a commonly observed characteristic of rules that are built on a single sample, and are prone to overfitting to the incidentals of that sample and not generalising to variants (Canfora et al., 2020; Coscia et al., 2025). The third incorporates detection-oriented guidance to combine multiple indicators from a rule occurring at the same time, and to prioritize precision over recall, as per Bilot et al., 2026, and Canfora et al., 2020, where they formalized the property of a rule, called looseness, that it produces false positives; the latter are reduced by requiring multiple indicators of a rule to co-occur in their condition, rather than matching on a generic pattern. The behavioural content is the same for all three prompts, meaning the behaviour name, type and description do not change between the three prompts and any difference in output quality between the adjacent prompts can be attributed to the one layer of guidance that was added. The prompts were not created following a formalised template but rather based on this reasoning, and are used as a controlled instrument to test the effect of prompt detail rather than an exhaustive exploration of the possible prompting strategies.

3.5 Experimental Procedure

Two experiments were conducted, one of which was evaluated on the entire corpus. The first (RQ1 and RQ3) led to the creation of rules for all fifteen behaviours for each of the five models, employing the third prompt. The second (RQ2) produced rules for all 15 behaviours with the model that was most often compiled in the first experiment with each one of the three prompts. Each rule has been initially tested by compiling it. The rules that compiled were then merged together by

model, per prompt, and scanned as a package together, so as long as one of the rules in the package matched it, it was considered detected. To reduce the randomness between runs, Generation used a temperature of zero.

3.6 Evaluation Metrics

Compile rate: Percentage of the rules generated that compile as valid YARA rules. Detection rate and false-positive rate are the shares of malicious and benign packages that a rule set matches, and are used as the main measures because they are easy to interpret. Detection rate is equal to recall for the entire corpus. It is informative and is reported with them at about the two-to-one class ratio that is also affected by the imbalance. Note that detection is computed over the entire corpus and not by behaviour as the data does not contain information about the behaviour exhibited by each package, and thus detection scores represent the overall match rate and not the behaviour-specific recall.

4. Design Specification

The system is divided into two stages: a generation stage to generate YARA rules from behaviour description by local LLMs and an evaluation stage to test the generated rules against package corpus. The overall flow is shown in figure 1.

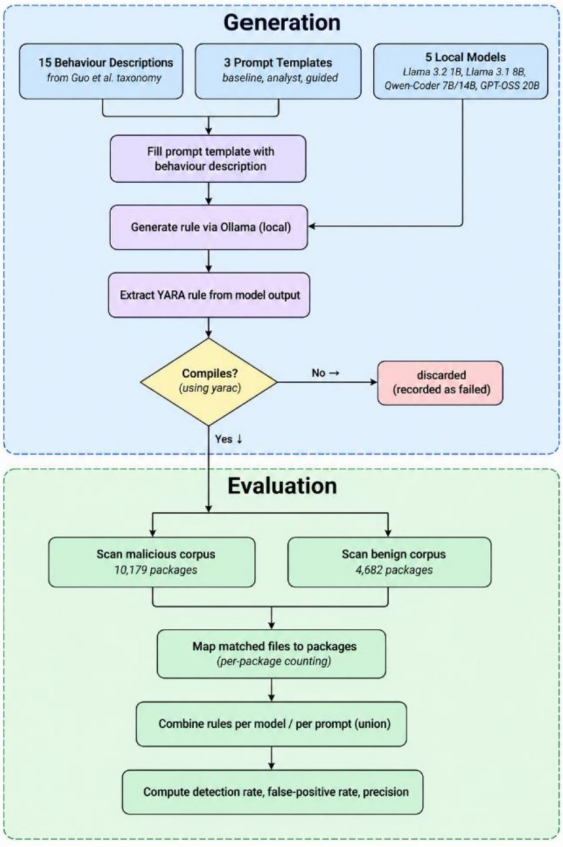


Figure 1

4.1 Generation Stage

The generation stage requires 3 hard coded inputs: list of 15 behaviour descriptions, prompt templates, and a list of models. The description is added to the prompt template for each model, prompt and behaviour, and then passed to the model via Ollama that is running the models locally. The model returns text which includes a YARA rule, typically in conjunction with other text which may include explanations. This text is fed into a parsing step that removes the rule from it. This extracted rule is then piped to the YARA compiler, where it is verified if it is valid. The rules and whether or not it compiled are recorded. By fixing the inputs and separating them from the code, the only changes between runs will be the model and the prompt, allowing any difference in output to be attributed to these.

4.2 Evaluation Stage

The evaluation stage is concerned with the rules that have been compiled and testing them against the corpus. Each rule is scanned recursively over the malicious packages and, separately, over the benign packages. A step maps each matching file back to its package, and a rule reports matches at the file level, therefore, results are reported per package, not per file. To evaluate a model or a prompt, all the packages covered by all the matching rules are added together, the packages are detected if any of the rules detect them. These numbers are used to calculate detection rate, false-positive rate and precision for this system.

4.3 Design Choices

There are three options that determine the design. First, the models are run locally, not via a cloud service, which means that the malware remains on the analysts machine and thus answers the data-governance concern that motivates the work. Second, rules are not generated from malware samples, and are not dependent on the specific malware that they were generated from. Third, evaluation is conducted on the level of the package, since a bad package typically has one bad file among several normal ones, and the total of the bad files would be underestimated if the evaluation were done on the level of the individual file.

4.4 Reproducibility

Each stage is a script in the design, the behaviour descriptions and prompts are hardcoded rather than programmed into the code that would use them. This allows the generation and evaluation to be repeated and verified, and the same inputs would result in the same rules and outputs.

5. Implementation

This pipeline is implemented in Python, and requires the use of 2 external tools: Ollama for running the language models locally and YARA for compiling and running the rules. The behaviour descriptions, prompt templates and model list were represented as independent configuration files and each part of the pipeline was implemented as a script that reads them.

5.1 Generation

A script that iterates over each model, each prompt and each behaviour was used for generation. For each combination, it fills in the prompt and feeds it into the model via Ollama's local interface. The text that the model returns is typically a rule along with more text; a parsing step is used to find the rule within the reply. The extracted rule is saved in the file that is named after the model, prompt and behaviour and then submitted to the YARA compiler to see if the rule is valid. If it is mandatory to record each rule compiled in a log. Generation was run at a temperature of zero.

5.2 Corpus Preparation

The malicious packages were provided as compressed archive files and were unpacked in a folder of packages prior to scanning. The benign packages were downloaded from PyPI, and extracted in the same manner, thus providing the same folder structure for both corpora. The same scanning code was operated on both of the following structures.

5.3 Evaluation

Scripts were used to evaluate them by traversing a set of rules over the two corpora. The malicious packages and benign packages are both passed to each rule that compiled is executed, in turn. A step associates each match file with a package, thus reporting matches per package. Rules that apply to a model, or for a prompt, are merged such that a package is counted once if it matches any of the rules. The scripts calculate detection rate, false-positive rate, and precision based on the results and output them to a results file.

5.4 Output

The first experiment was using 75 rules generated by the pipeline (five models x 15 behaviours) and the second experiment was using a second set of rules generated by the same pipeline (the selected model x three prompts x 15 behaviours). The results of the compilation of each rule and the per-package scan results for each rule were noted, which are reported in Section 6.

6. Evaluation

The results of the two experiments are reported in this section. The capability of the model is discussed in Section 6.1, the influence of prompt detail in Section 6.2, and the comparison with the hand-written baseline in Section 6.3. The results are discussed together in section 6.4.

6.1 Model Capability (RQ1)

There were two aspects of capability measured: whether the model could produce rules that were valid, and how well the model's rules did.

Table 1 below represents the percentage of models that compiled when using the guided prompt for the fifteen behaviours.

Model	Type	Rules compiled
Llama 3.2 (1B)	general-purpose	0 / 15
Llama 3.1 (8B)	general-purpose	1 / 15
Qwen2.5-Coder (7B)	code-specialised	9 / 15
Qwen2.5-Coder (14B)	code-specialised	11 / 15
GPT-OSS (20B)	general-purpose	4 / 15

Table 1. Compile rate (guided prompt, 15 behaviours).

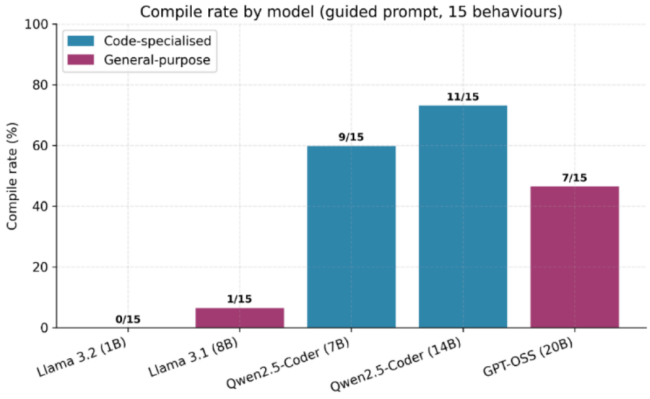


Figure 2 Compile rate by model, guided prompt (15 behaviours).

Rate of compilation did not vary with size. The two code-tuned models had a much higher number of valid rules (9 and 11 of 15) than the general purpose models, including the largest of the latter, GPT-OSS (20B), which generated just 4. None of the 1B model results was valid YARA, with just text that looked like a rule. This indicates that the code was more important than size in tuning for a valid YARA. The second effect was a secondary one where the larger family – 14B – had slightly more valid rules than the smaller family – 7B – giving a smaller effect of size.

Model	Rules	Detection	False positive	Precision
Llama 3.2 (1B)	0	0.0%	0.0%	–
Llama 3.1 (8B)	1	91.6%	16.9%	0.922
Qwen2.5-Coder (7B)	9	73.7%	25.4%	0.863
Qwen2.5-Coder (14B)	11	85.0%	22.6%	0.891
GPT-OSS (20B)	4	7.9%	9.6%	0.640
Baseline (hand-written)	**15**	**95.6%**	**19.2%**	**0.915**

Table 2. Precision by model, detection, and false positive rate (balanced corpus, 10179 malicious / 4682 benign).

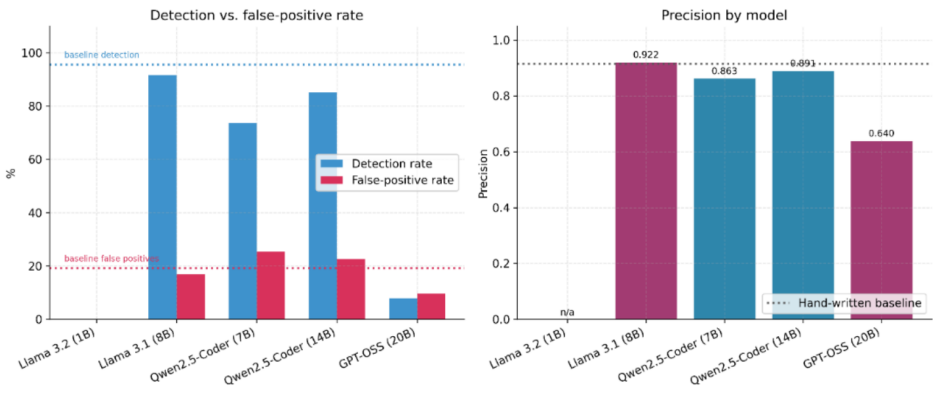


Figure 1. Detection rate, false-positive rate, and precision by model, with hand-written baseline.

The 1B model didn't detect anything since it did not generate any valid rules. The largest model, GPT-OSS, was the least effective model, with only a small percentage (7.9%) identified by it. The 7B set was compared to the 14B set in the code-tuned models, and it was found that the 14B set

performed better than the 7B set with a similar false-positive rate (85.0% vs 73.7%) again suggesting a small advantage for size within the same family. The single rule from Llama 3.1 (8B) had high detection and precision, however, since it is a single rule only, it is not a reliable basis for comparison and is indicated with caution.

6.2 Effect of Prompt Detail (RQ2)

For the second experiment, the highest compile rate model, Qwen2.5-Coder (14B) was used in conjunction with all three prompts. The result are presented in Table 3.

Prompt	Detection	False positive	Precision
Baseline (P1)	96.5%	32.7%	0.865
Analyst (P2)	86.1%	26.5%	0.876
Guided (P3)	85.0%	28.7%	0.866

Table 3. Detection, false positive rate, and precision, according to prompt (Qwen2.5-Coder 14B).

The false-positive rate was lower for adding detail to the prompt than for the baseline prompt (26.5% and 28.7% compared to 32.7%), and the guided prompt (with detail) resulted in the detection of a slightly smaller number of malicious packages. There was a minor difference between the two types of analyst and guided prompts, and no improvement was evident with the guided prompt as this had a slightly higher false-positive rate. There was very little difference in precision for the three prompts (from 0.865 to 0.876). The results in this study indicate that providing some guidance to the model beyond a simple request helps to decrease the number of false positives, but adding more detail did not result in a significant benefit in this study.

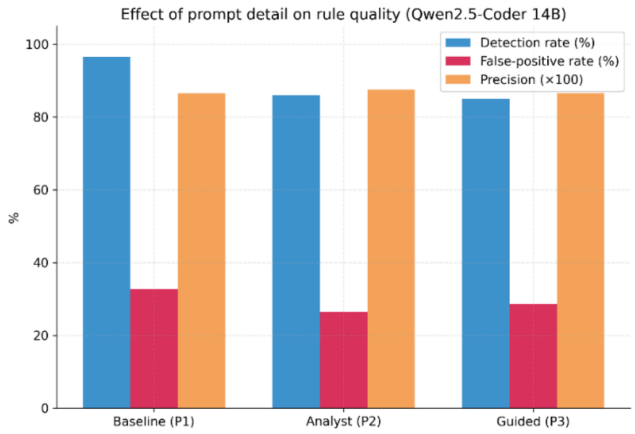


Figure 2 Effect of prompt detail on rule quality, Qwen2.5-Coder (14B).

6.3 Comparison with the Hand-Written Baseline (RQ3)

The hand-written baseline rules were scanned in the same way as the generated rules, to provide a comparison.

All the rules were scanned as the generated rules. The baseline achieved a precision of 0.915, a false-positive rate of 19.2%, and 95.6% detection rate (Table 2). No rule set with the generated rules matched this. The best generated set from Qwen2.5-Coder (14B) achieved 85.0% detection with a 22.6% false-positive rate, and a precision of 0.891, which is near to the precision but lower in the detection rate and higher in the false-positive rate. The rules that are created thus identified a high percentage of malicious packages and were not as high as the hand-written rules.

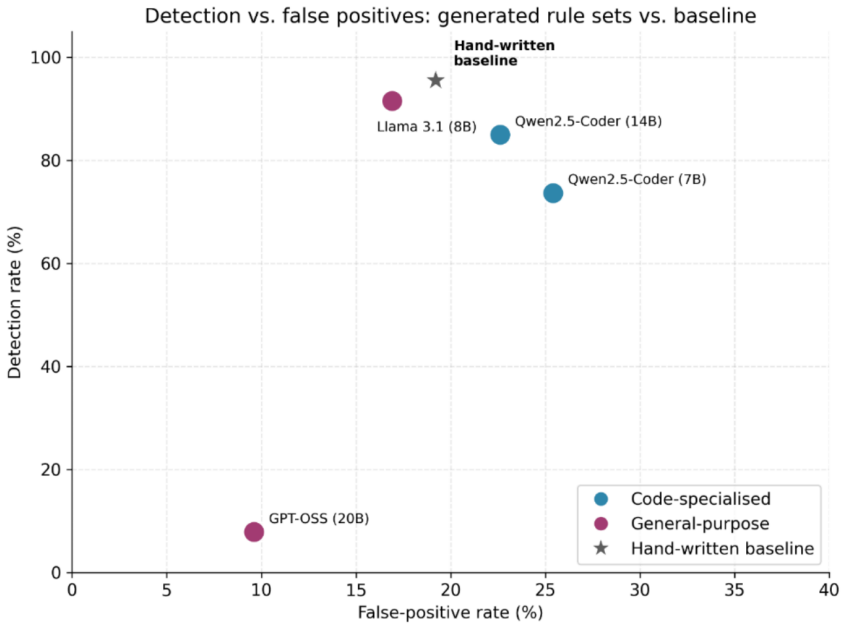


Figure 3. Detection rate vs. false-positive rate: generated rule sets vs. hand-written baseline

6.4 Discussion

All the results are put together to give a consistent picture. With valid rules, code tuning was the most powerful factor: the code-tuned models yielded many more rules that were valid than the general-purpose models did, even the larger ones. Size had a smaller effect, but was only apparent within the same family, where the larger code model outperformed the smaller model on the detection rate, but not on the compile rate. This finds agreement with previously found tuned smaller models outperforming larger general models in security tasks (Levi et al., 2026).



Figure 4. Model size vs. detection rate, by tuning type.

As far as effectiveness, the rules generated were useful but not exactly the same as the hand-written baseline rules. The best generated set was able to detect the vast majority of malicious packages with the precision nearly equal to the baseline, which is able to detect more at a lower false positive rate. This is in line with previous studies that showed auto-generated rules are less powerful than hand-written ones (Raff et al., 2020), and that it shows that on this dataset, small local models can support the task of hand-writing rules but not perform it.

Results from the prompts indicate that providing some guidance is beneficial by lowering the number of false positives, but that there was little additional benefit when the guidance was more

than a simple prompt for this model. This should be interpreted carefully, and is only based on one model and 15 rules per prompt.

Inspecting each of the individual rules reveals that detection was concentrated on a handful of rules, rather than across all the behaviours. A few dozen rules matched a few hundred or less malicious packages, while a few thousand matched for the two code-tuned models. For Qwen2.5-Coder (14B), the rules for 2 behaviours were matched to 6,643 and 7,959 packages, respectively.

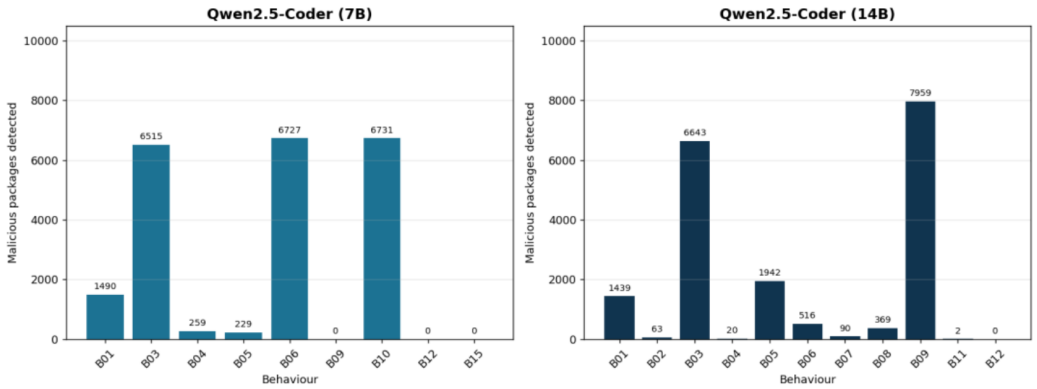


Figure 7. Per-rule detection for Qwen2.5-Coder 7B and 14B. Number of malicious packages matched by each compiled rule, by behaviour.

while for Qwen2.5-Coder (7B), there were three rules each matched more than 6,500 packages. The rest of the rules contributed somewhat less and some of the rules compiled were not found in any packages. The single rule which matched 9,325 packages on its own accounts for much of the high aggregate detection score, yet also illustrates why a single score is not a good basis for comparison. There were a few rules that matched the packages in GPT-OSS (20B), but each matched comparatively few. The pattern suggests that the overall detection figure is what has been impacted by the few rules that are broad, and not by consistent detection of behaviours. It should be interpreted as a number of packages matched, not as a description of the accuracy of each behaviour, as there is no label data to identify which behaviour is represented by each package.

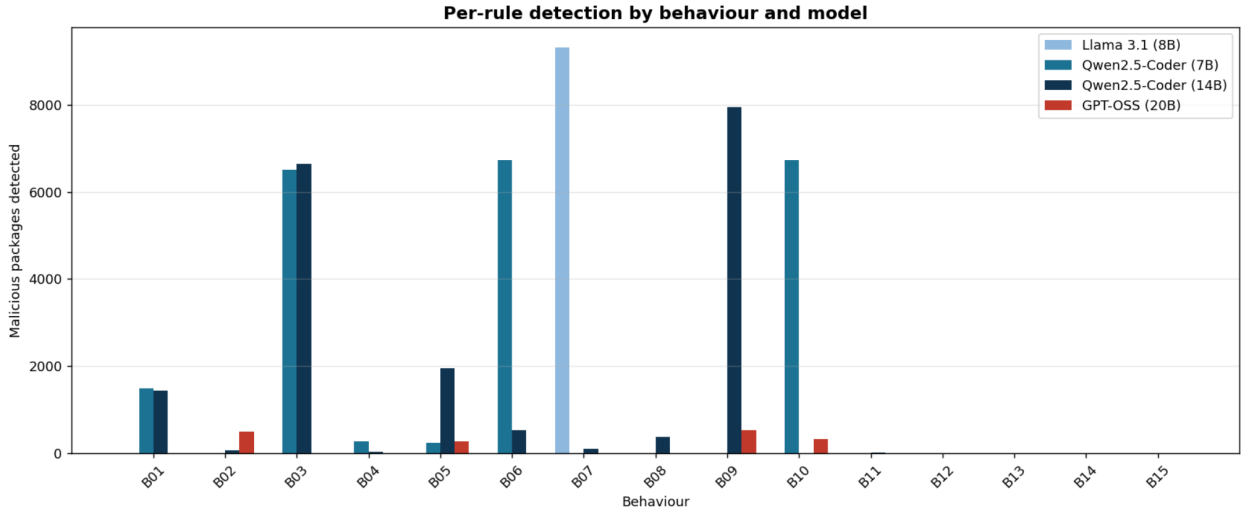


Figure 8. Malicious packages matched per rule, by behaviour and model..

Several limitations apply. Because the models vary in family, training data and tuning simultaneously, it is not possible to separate size and tuning; comparisons are only possible within-family and same-size models. However, detection is not per behaviour, but is actually over the entire corpus, therefore it reflects overall match rate, not behaviour-specific recall. There are a relatively small number of rules per model, especially for those that generated only a few valid rules, and this restricts the significance of the effectiveness measurements for those models.

7. Conclusion and Future Work

This study looked at whether small, locally-run LLMs can generate useful YARA rules for detecting malicious PyPI packages. Local models were used to address the data-governance problem that arises when malware is sent to a cloud service. Five open models generated rules for fifteen categories of malicious behaviour, and the rules were tested against a set of malicious and benign packages and compared with a set of hand-written rules.

For the first question, of how model capability affects rule generation, the main factor was whether a model was tuned for code, not its size. The code-tuned models produced far more valid rules than the general-purpose models, including ones that were larger, and detected more malicious packages. Size had a smaller effect that was only clear within the same model family, where the larger code model did slightly better than the smaller one. For the second question, of prompt detail, giving the model some guidance beyond a bare request reduced false positives, but adding more detail than that gave little further benefit for the model tested. For the third question, the generated rules detected a large share of malicious packages but did not match the hand-written rules, which detected more with fewer false positives.

Overall, the results show that small, locally-run models can help with writing YARA rules for this task but cannot yet replace hand-written rules, and that their output still needs to be checked by an analyst. The approach is still useful, because it keeps the malware on local hardware and produced reasonable rules from behaviour descriptions alone.

Several limitations apply. The five models differ in family, training data, and tuning at the same time, so size and tuning cannot be fully separated except in the within-family and same-size comparisons. Detection was measured across the whole corpus rather than per behaviour, because the dataset does not record which behaviour each package shows, so the figures reflect the overall match rate rather than behaviour-specific detection. Some models produced only a few valid rules, which limits how much can be read into their results. The prompt comparison was based on one model.

Future work could build on this in three ways. First, comparing similarly sized models from different families would show whether the model family affects rule generation on its own, separately from size. Second, a dataset that labels which behaviour each package shows would allow detection to be measured for each behaviour, showing which behaviours small models handle well and which produce weak or noisy rules. Third, testing the generated rules on PyPI malware released after the models' training period would show whether they detect genuinely new threats or only recognise patterns already seen during training, which matters because the goal of detection is to catch new attacks.

References

Coscia, A., Lorusso, R., Maci, A. and Urbano, G. (2025) 'APIARY: An API-based automatic rule generator for YARA', Computers & Security, 153, 104397. DOI: 10.1016/j.cose.2025.104397

Gao, X., Sun, X., Cao, S., Huang, K., Wu, D., Liu, X., Lin, X. and Xiang, Y. (2025) 'MalGuard: Towards Real-Time, Accurate, and Actionable Detection of Malicious Packages in PyPI Ecosystem', 34th USENIX Security Symposium, pp. 4741-4758. Available at: https://arxiv.org/abs/2506.14466

Guo, W., Xu, Z., Liu, C., Huang, C., Fang, Y. and Liu, Y. (2023) 'An Empirical Study of Malicious Code in PyPI Ecosystem', 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). Available at: https://arxiv.org/abs/2309.11021

Ladisa, P., Ponta, S.E., Ronzoni, N., Martinez, M. and Barais, O. (2023a) 'On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI', 39th Annual Computer Security Applications Conference (ACSAC), pp. 71-82. DOI: 10.1145/3627106.3627138

Ladisa, P., Plate, H., Martinez, M. and Barais, O. (2023b) 'SoK: Taxonomy of Attacks on Open-Source Software Supply Chains', 2023 IEEE Symposium on Security and Privacy (SP), pp. 1509-1526. DOI: 10.1109/SP46215.2023.10179304

Levi, M., Ohayon, D., Blobstein, A., Sagi, R., Molloy, I. and Allouche, Y. (2026) 'Toward Cybersecurity-Expert Small Language Models', International Conference on Machine Learning (ICML). Available at: https://arxiv.org/abs/2510.14113

Li, N., Wang, S., Feng, M., Wang, K., Wang, M. and Wang, H. (2023) 'MalWuKong: Towards Fast, Accurate, and Multilingual Detection of Malicious Code Poisoning in OSS Supply Chains', 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1993-2005. DOI: 10.1109/ASE56229.2023.00073

Liang, W., Ling, X., Wu, J., Luo, T. and Wu, Y. (2023) 'A Needle is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering', 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 307-318.

Mansour, Z., Ou, W., Ding, S.H.H., Zulkernine, M. and Charland, P. (2025) 'NeuroYara: Learning to Rank for Yara Rules Generation Through Deep Language Modeling and Discriminative N-Gram Encoding', IEEE Transactions on Dependable and Secure Computing, 22(2), pp. 1747-1762. DOI: 10.1109/TDSC.2024.3449641

Patsakis, C., Casino, F. and Lykousas, N. (2024) 'Assessing LLMs in Malicious Code Deobfuscation of Real-world Malware Campaigns', Expert Systems with Applications, 256, 124912. DOI: 10.1016/j.eswa.2024.124912

Raff, E., Zak, R., Munoz, G.L., Fleming, W., Anderson, H.S., Filar, B., Nicholas, C. and Holt, J. (2020) 'Automatic Yara Rule Generation Using Biclustering', 13th ACM Workshop on Artificial Intelligence and Security (AISec). DOI: 10.1145/3411508.3421372

Sejfia, A. and Schäfer, M. (2022) 'Practical Automated Detection of Malicious npm Packages', 44th International Conference on Software Engineering (ICSE), pp. 1681-1692. DOI: 10.1145/3510003.3510104

Toda, T. and Mori, T. (2025) 'CHASE: LLM Agents for Dissecting Malicious PyPI Packages', 2nd IEEE/ACM International Conference on AI-powered Software (AIware), pp. 1-10. DOI: 10.1109/AIware69974.2025.00008

Yan, J. and Zhao, B. (2025) 'PyPIMalDet: A malicious PyPI package detection method combining code features and metadata features', Neural Networks, 197, 108487. DOI: 10.1016/j.neunet.2025.108487

Zahan, N., Burckhardt, P., Lysenko, M., Aboukhadijeh, F. and Williams, L. (2025) 'Leveraging Large Language Models to Detect NPM Malicious Packages', 47th IEEE/ACM

International Conference on Software Engineering (ICSE), pp. 2625-2637. DOI: 10.1109/ICSE55347.2025.00146

Zhang, J., Huang, K., Huang, Y., Chen, B., Wang, R., Wang, C. and Peng, X. (2025) 'Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence', ACM Transactions on Software Engineering and Methodology, 34(4), pp. 104:1-104:28. DOI: 10.1145/3705304

Zhang, X., Chen, H., He, Y., Niu, W. and Li, Q. (2025) 'Automatically Generating Rules of Malicious Software Packages via Large Language Model', 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp. 734-747. Available at: https://arxiv.org/abs/2504.17198

Bilot, T., El Madhoun, N., Al Agha, K. and Zouaoui, A. (2026) 'Automating the detection of evasive Windows malware: an evaluated YARA rule library for anti-VM and anti-sandbox techniques', *Systems*, 6(2), p. 69. Available at: https://doi.org/10.3390/systems6020069

Canfora, G., Carapella, M., Del Vecchio, A., Nardi, L., Pirozzi, A. and Visaggio, C.A. (2020) 'About the robustness and looseness of YARA rules', in Casola, V., De Benedictis, A. and Rak, M. (eds) *Testing Software and Systems: ICTSS 2020*, Lecture Notes in Computer Science, vol. 12543. Cham: Springer, pp. 104–120. Available at: https://doi.org/10.1007/978-3-030-64881-7_7

Coscia, A., Lorusso, R., Maci, A. and Urbano, G. (2025) 'APIARY: an API-based automatic rule generator for YARA to enhance malware detection', *Computers and Security*, 153, p. 104397. Available at: https://doi.org/10.1016/j.cose.2025.104397